

SQL PLACEMENT CODING NOTES

25 High-Frequency SQL Problems — Detailed Beginner-to-Placement Guide

How to use: First read the problem and try it yourself. Then check the query. Understand why each clause is used. Finally, close the notes and rewrite the query from memory.

Important: SQL syntax can differ slightly between MySQL, PostgreSQL, Oracle and SQL Server. The examples use common SQL patterns; date/limit syntax may need adjustment for your DBMS.

COMMON TABLES USED IN THESE PROBLEMS

Employee	Department	Customers / Orders
emp_id, name, department, salary, manager_id	dept_id, dept_name	customer_id, name / order_id, customer_id, order_amount

Typical Employee data:

emp_id	name	department	salary	manager_id
1	Asha	IT	90000	NULL
2	Ravi	IT	80000	1
3	Kiran	HR	70000	NULL
4	Meena	HR	70000	3
5	John	IT	60000	1

■ **Core patterns you will repeatedly use:** MAX/MIN/AVG → aggregate; GROUP BY/HAVING → group problems; JOIN → related tables; LEFT JOIN + IS NULL → missing records; subquery → compare with calculated value; DENSE_RANK → Nth/top salary levels; SELF JOIN → employee-manager.

SALARY & RANKING PROBLEMS

1. Find Second Highest Salary

Concept: MAX + subquery

Query:

```
SELECT MAX(salary) AS second_highest
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);
```

How it works: The inner MAX finds the highest salary. The WHERE condition removes that salary. The outer MAX then finds the largest salary remaining.

Placement tip/trap: This returns the second *distinct* salary. With salaries 90000, 80000, 80000, the answer is 80000.

2. Find Third Highest Salary

Concept: Nested subquery / distinct salary levels

Query:

```
SELECT MAX(salary) AS third_highest
FROM Employee
WHERE salary < (
  SELECT MAX(salary)
  FROM Employee
  WHERE salary < (SELECT MAX(salary) FROM Employee)
);
```

How it works: The innermost MAX finds the first level. The middle query finds the maximum below it (second level). The outer query finds the maximum below the second level (third level).

Placement tip/trap: This version works with distinct salary levels. DENSE_RANK is usually easier when you need a general Nth-highest pattern.

3. Find Nth Highest Salary

Concept: DENSE_RANK + subquery

Query:

```
SELECT salary
FROM (
  SELECT salary,
  DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM Employee
) x
WHERE rnk = 3;
```

How it works: DENSE_RANK gives rank 1 to the highest distinct salary, rank 2 to the next, and so on. Replace 3 with N.

Placement tip/trap: DENSE_RANK handles ties without skipping the next distinct salary rank.

4. Find Duplicate Records

Concept: GROUP BY + HAVING

Query:

```
SELECT name, department, salary, COUNT(*) AS cnt
FROM Employee
GROUP BY name, department, salary
HAVING COUNT(*) > 1;
```

How it works: Rows having the same selected column combination form one group. COUNT tells how many times that combination occurs. HAVING keeps only groups appearing more than once.

Placement tip/trap: Choose the columns that define a duplicate. Do not automatically group by every column if the question defines duplicates using only some fields.

5. Remove Duplicate Records

Concept: ROW_NUMBER + duplicate identification

Query:

```
SELECT *
FROM (
  SELECT e.*,
  ROW_NUMBER() OVER (
    PARTITION BY name, department, salary
    ORDER BY emp_id
  ) AS rn
  FROM Employee e
) x
WHERE rn > 1;
```

How it works: PARTITION BY creates one group for each duplicate combination. ROW_NUMBER marks the first row as 1 and later copies as 2, 3, etc. The SELECT above identifies rows to remove.

Placement tip/trap: Always test the SELECT first. Actual DELETE syntax differs by DBMS. The safe conceptual rule is: keep rn = 1 and remove rn > 1.

6. Employees Earning Above Average

Concept: Subquery + AVG

Query:

```
SELECT *
FROM Employee
WHERE salary > (SELECT AVG(salary) FROM Employee);
```

How it works: The inner query calculates the overall average salary. The outer query compares each employee's salary with that single value.

Placement tip/trap: Remember: aggregate first, compare second.

7. Employees Earning Maximum Salary

Concept: MAX + subquery

Query:

```
SELECT *
FROM Employee
WHERE salary = (SELECT MAX(salary) FROM Employee);
```

How it works: MAX finds the highest salary. Equality returns every employee having that salary, including ties.

Placement tip/trap: Do not use LIMIT 1 if the question asks for all employees sharing the maximum.

8. Maximum Salary Department-wise

Concept: GROUP BY + MAX

Query:

```
SELECT department, MAX(salary) AS max_salary
FROM Employee
GROUP BY department;
```

How it works: GROUP BY creates one group per department. MAX is calculated separately inside each department.

Placement tip/trap: This returns the maximum salary value, not necessarily the employee name. To get the employee too, use a ranking/window solution or another join/subquery.

9. Second Highest Salary Department-wise

Concept: DENSE_RANK + PARTITION BY

Query:

```
SELECT department, name, salary
FROM (
  SELECT department, name, salary,
  DENSE_RANK() OVER (
    PARTITION BY department
    ORDER BY salary DESC
  ) AS rnk
FROM Employee
) x
WHERE rnk = 2;
```

How it works: PARTITION BY starts a separate ranking for every department. ORDER BY salary DESC puts the highest salary first. rnk = 2 selects the second distinct salary level in each department.

Placement tip/trap: This is a key placement pattern: **department-wise ranking = PARTITION BY department**.

10. Count Employees Department-wise

Concept: GROUP BY + COUNT

Query:

```
SELECT department, COUNT(*) AS employee_count
FROM Employee
GROUP BY department;
```

How it works: Rows are grouped by department and COUNT(*) counts the rows in each group.

Placement tip/trap: If NULL department rows should be excluded, add an appropriate WHERE condition.

11. Departments With More Than 5 Employees

Concept: GROUP BY + HAVING

Query:

```
SELECT department, COUNT(*) AS employee_count
FROM Employee
GROUP BY department
HAVING COUNT(*) > 5;
```

How it works: COUNT is calculated per department. HAVING then filters the groups whose count is greater than 5.

Placement tip/trap: Use HAVING, not WHERE, for a condition on COUNT.

12. Employees Without Departments

Concept: IS NULL

Query:

```
SELECT *
FROM Employee
WHERE department IS NULL;
```

How it works: NULL means the value is missing/unknown. IS NULL is used to test it.

Placement tip/trap: Do not write department = NULL.

13. Departments Without Employees

Concept: LEFT JOIN + IS NULL

Query:

```
SELECT d.*
FROM Department d
LEFT JOIN Employee e
ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;
```

How it works: LEFT JOIN preserves every department. A department with no matching employee gets NULL on the Employee side. Filtering e.emp_id IS NULL leaves only departments with no employees.

Placement tip/trap: This is one of the most important 'find missing records' patterns.

14. Employees Whose Salary Is Greater Than Their Manager

Concept: SELF JOIN

Query:

```
SELECT e.name AS employee,
e.salary,
m.name AS manager,
m.salary AS manager_salary
FROM Employee e
JOIN Employee m
ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

How it works: The Employee table is used twice: e represents the employee and m represents the manager. The SELF JOIN connects employee.manager_id to manager.emp_id. WHERE compares their salaries.

Placement tip/trap: Know SELF JOIN well; this is a common interview pattern.

15. Employees Who Joined in a Particular Year

Concept: Date function

Query:

```
SELECT *
FROM Employee
WHERE YEAR(join_date) = 2024;
```

How it works: YEAR extracts the year portion of the date and WHERE keeps rows matching 2024.

Placement tip/trap: YEAR() is MySQL-style/common syntax. Oracle, PostgreSQL and SQL Server have different date functions, so learn the equivalent for the DBMS used in the test.

16. Find Duplicate Salaries

Concept: GROUP BY + HAVING

Query:

```
SELECT salary, COUNT(*) AS cnt
FROM Employee
GROUP BY salary
HAVING COUNT(*) > 1;
```

How it works: Each salary becomes a group. Salaries appearing more than once survive the HAVING condition.

Placement tip/trap: Unlike duplicate records, here only the salary value defines the duplicate.

17. Find Top 3 Salaries

Concept: DISTINCT + ORDER BY + LIMIT

Query:

```
SELECT DISTINCT salary
FROM Employee
ORDER BY salary DESC
LIMIT 3;
```

How it works: DISTINCT creates distinct salary levels. DESC puts the largest first. LIMIT 3 keeps the first three salary levels.

Placement tip/trap: If the question means top 3 employees rather than top 3 distinct salary values, do not use DISTINCT.

18. Top 3 Salaries Department-wise

Concept: DENSE_RANK + PARTITION BY

Query:

```
SELECT department, name, salary
FROM (
  SELECT department, name, salary,
  DENSE_RANK() OVER (
    PARTITION BY department
    ORDER BY salary DESC
  ) AS rnk
  FROM Employee
) x
WHERE rnk <= 3;
```

How it works: Ranking restarts for every department. rnk <= 3 returns the top three distinct salary levels within each department.

Placement tip/trap: DENSE_RANK can return more than three employees if there are ties. That is usually correct when 'top 3 salaries' means salary levels.

19. Find Consecutive Records

Concept: LAG / previous row

Query:

```
SELECT emp_id,
LAG(emp_id) OVER (ORDER BY emp_id) AS previous_id
FROM Employee;
```

How it works: LAG gives the value from the previous row according to the chosen ordering. You can then compare the current ID with previous_id to detect consecutive values.

Placement tip/trap: 'Consecutive' must be defined: consecutive IDs, dates, login days, etc. The ordering and comparison depend on the question.

20. Customers Who Never Ordered

Concept: LEFT JOIN + IS NULL

Query:

```
SELECT c.*
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

How it works: LEFT JOIN keeps all customers. Customers without an order have NULL in the Orders columns, so IS NULL identifies them.

Placement tip/trap: NOT EXISTS is another correct approach.

21. Customers Who Ordered More Than Once

Concept: GROUP BY + HAVING

Query:

```
SELECT customer_id, COUNT(*) AS order_count
FROM Orders
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

How it works: Orders are grouped by customer. HAVING keeps customers with more than one order.

Placement tip/trap: If you need customer names, JOIN this result to Customers or group after joining.

22. Products Never Purchased

Concept: LEFT JOIN + IS NULL

Query:

```
SELECT p.*
FROM Products p
LEFT JOIN OrderItems oi
ON p.product_id = oi.product_id
WHERE oi.product_id IS NULL;
```

How it works: Every product is preserved by the LEFT JOIN. Products without an OrderItems match have NULL on the right and are selected.

Placement tip/trap: This is the same missing-record pattern as 'customers who never ordered'.

23. Highest Order Amount

Concept: MAX

Query:

```
SELECT MAX(order_amount) AS highest_order_amount
FROM Orders;
```

How it works: MAX returns the largest order amount.

Placement tip/trap: If asked for the complete order row, MAX alone is not enough; use a subquery/ranking approach to return the corresponding order.

24. Average Salary Department-wise

Concept: GROUP BY + AVG

Query:

```
SELECT department,
AVG(salary) AS avg_salary
FROM Employee
GROUP BY department;
```

How it works: GROUP BY creates department groups and AVG calculates the average salary in each group.

Placement tip/trap: Remember: department-wise calculation usually means GROUP BY department.

25. Employees Sharing the Same Salary

Concept: Subquery + GROUP BY + HAVING

Query:

```
SELECT *
FROM Employee
WHERE salary IN (
  SELECT salary
  FROM Employee
  GROUP BY salary
  HAVING COUNT(*) > 1
);
```

How it works: The inner query finds salary values that occur more than once. The outer query returns every employee whose salary belongs to that duplicate-salary set.

Placement tip/trap: This is different from finding duplicate records: employees can be different but share the same salary.

FINAL REVISION — THE PATTERNS TO MEMORIZE

Question type	Main pattern
2nd / 3rd / Nth highest	DENSE_RANK or MAX + subquery
Above average	WHERE salary > (SELECT AVG(...))
Maximum	MAX + equality/subquery
Duplicate values	GROUP BY + HAVING COUNT(*) > 1
Department-wise	GROUP BY department
Department-wise top/Nth	DENSE_RANK + PARTITION BY
More than N rows per group	GROUP BY + HAVING COUNT(*) > N
No related record	LEFT JOIN + IS NULL / NOT EXISTS
Employee vs manager	SELF JOIN
Particular year	Date extraction/function
Top N distinct values	DISTINCT + ORDER BY + LIMIT
Consecutive rows	LAG/LEAD + ordering

SQL QUERY-BUILDING CHECKLIST

Before writing a query, ask yourself:

1. **What table?** → FROM
2. **What columns?** → SELECT
3. **Filter individual rows?** → WHERE
4. **Need another table?** → JOIN
5. **Need groups?** → GROUP BY
6. **Filter groups?** → HAVING
7. **Need a calculated value first?** → Subquery
8. **Need ranking?** → Window function
9. **Need missing matches?** → LEFT JOIN + IS NULL / NOT EXISTS

■ **Final target:** You do not need to memorize 25 unrelated answers. Learn these recurring patterns and you can build many placement queries from them.

AFTER FINISHING THIS PDF

Your SQL theory is already covered. After these 25 problems, the SQL portion should move into **practice mode**: solve the same questions without notes, then do mixed timed SQL questions. Avoid adding more theory unless you encounter a concept you genuinely cannot understand.